Relatório técnico — ASTRO BOT no AnyPS5 (Windows/AMD)

Data: 23/10/2026
Jogo: ASTRO BOT — `PPSA21567`, content version `01.018.000`
Base: AnyPS5 v0.1.3 (release) + libs compiladas da `main`
Objetivo: eliminar o crash de GPU que impedia o jogo de passar do carregamento inicial e documentar o procedimento para portes futuros.

---

1. Resumo executivo

- A v0.1.3 tentava criar um **heap mirror de 3.388.997.632 bytes (≈3,15 GiB)** em uma única alocação Vulkan.
- O driver AMD no Windows reporta `maxMemoryAllocationSize = 0x80000000` (**2 GiB**). A alocação falhava com `VK_ERROR_OUT_OF_DEVICE_MEMORY` (`Vulkan result -2`) e o processo abortava.
- A `main` do AnyPS5 já contém a correção:
- **PR #7292** — importa faixas registradas maiores que o limite do dispositivo **em pedaços**;
- **PR #7647** — faz o heap mirror abranger apenas as páginas que o guest realmente escreveu.
- Compilei a `main` no commit `a0fb7c26a7969d0ae2b18aee923e43a30d133898` e substituí **o conjunto completo de libs** em `D:\AnyPS5-AstroBot\work\libs`.
- Resultado: o jogo passou do ponto do crash, carregou o nível `title_controller_ship`, apresentou frames em **2201×1238** e permaneceu rodando por **180 s** sem erro fatal. O teste foi encerrado manualmente; não houve crash.
- **Lição principal:** não basta copiar apenas `libSceAgcDriver.prx`. Os PRXs são construídos e carregados como um conjunto; misturar libs de versões diferentes gera `GetLastError 127` (procedimento não encontrado). Copie todas as libs do mesmo build.

---

2. Ambiente usado

Item	Valor
Sistema	Windows 11 IoT Enterprise LTSC, 10.0.26100, 64 bits
CPU	Intel Core i7-7700, 4 núcleos / 8 threads
RAM	16 GB
GPU	AMD Radeon RX 9060 XT, 16 GB
Driver AMD	`32.0.32015.7029`
Vídeo	3840×2160; o jogo apresentou em 2201×1238
Emulador base	AnyPS5 v0.1.3 (release)
Jogo	ASTRO BOT `PPSA21567`, content `01.018.000`
Executável	`D:\AnyPS5-AstroBot\work\app.exe`
Libs em uso	`D:\AnyPS5-AstroBot\work\libs`
Fonte compilada	`D:\AnyPS5-main` (commit `a0fb7c2`)
Build	`D:\AnyPS5-main\build`
GCC	WinLibs `x86_64-ucrt-posix-seh` 15.2.0 r7
CMake	4.3.1 do WinLibs no build (o sistema também tem 4.4.4)
Ninja	1.13.2
Python	3.12.0
Git	PortableGit 2.55.0.windows.5
Tamanho do fonte	≈1,2 GB
Tamanho do build	≈0,82 GB
Tempo de configure	≈107 s
Tempo de compilação	≈16 min (2506 etapas, `--parallel 8`)

> Observação: o `AdapterRAM` do WMI pode reportar 4 GB por limitação de 32 bits; o heap Vulkan device-local reportado no log é de **15.207 MiB**, compatível com a GPU de 16 GB.

---

3. Sintoma original

O jogo chegava a `Resident Load end`, `PlayGo finished` e criava texturas, mas abortava ao preparar um compute shader. Trecho exato de `D:\AnyPS5-AstroBot\work\run.v013.err.log:313`:

```
what():  compute shader 0x50057fb00: AGC graphics: heap mirror of 0x1000000000+0xca000000: vkAllocateMemory buffer: allocation=3388997632 memoryType=3 reclaimed=61865984 released=0 initialResult=-2 attempts=2; heap 0 328/15207 MiB; heap 1 4712/7505 MiB; heap 2 0/242 MiB: Vulkan result -2; heap mirrors hold 1439 MiB of 24576 MiB (APS5_HEAP_MIRROR_MIB); DISPATCH_INDIRECT on queue 0x0, arguments at 0x5074063c0, driver path gpu-side, read-ahead none
```

O processo terminava com exceção não tratada e `abort()`; no Windows, o código observado foi `0xc0000409` em `ucrtbase.dll`.

O que não era o problema

- Não era falta de VRAM: o heap 0 tinha **328 MiB usados de 15.207 MiB**.
- Não era memória host esgotada: o heap 1 tinha **4.712 MiB de 7.505 MiB**.
- Não era o limite total de memória, e sim o **limite por alocação** do driver AMD: **2 GiB**.

Por que as flags antigas não resolviam

`APS5_HEAP_MIRROR_MIB`, `APS5_HOST_IMPORT_MIB`, `APS5_ADDRESS_COPY_MAX_MIB`, `APS5_NO_LEASE_MIRROR` e `APS5_NO_WRITE_WATCH` não conseguem corrigir o caso porque qualquer caminho alternativo ainda precisava de **uma única alocação maior que 2 GiB**. O problema só é resolvido dividindo a faixa em pedaços menores que `maxMemoryAllocationSize`.

---

4. Causa raiz e correção upstream

Na v0.1.3, cada faixa registrada era importada como **um único host import**. Em placas AMD no Windows, `maxMemoryAllocationSize` é 2 GiB. Quando a faixa é maior, `vkAllocateMemory` falha; o fallback tenta criar um heap mirror do mesmo tamanho, que também falha, e o título aborta.

A `main` corrige isso em dois níveis:

- **PR #7292** — `fix(agc): import registered ranges larger than the device's largest allocation in pieces`
- https://github.com/boykopovar/AnyPS5/pull/7292
- Merge commit: `b061d2a621cf1a573532341197f4e3f04e05a085`
- Arquivos principais:
  - `core/libs/prx/libSceAgcDriver/Execution/src/VulkanDevice.cpp`
  - `core/libs/prx/libSceAgcDriver/Graphics/src/GuestBufferMemory.cpp`
  - `core/libs/prx/libSceAgcDriver/Graphics/include/GuestBufferMemory.hpp`
- A correção usa `Graphics::SetHostImportMaxBytes(maintenance3.maxMemoryAllocationSize)` e divide o import em `importPieces` menores ou iguais ao limite.
- **PR #7647** — `fix(agc): bind heap mirrors only where the guest holds bytes`
- https://github.com/boykopovar/AnyPS5/pull/7647
- Merge commit: `2f83cc782f5b71976f7a91fb58cb801177f2abd2`
- Reduz o heap mirror para as páginas efetivamente escritas pelo guest, em vez da faixa reservada inteira.

A v0.1.3 não contém `importMaxBytes`, `importPieces` nem `SetHostImportMaxBytes`. Também não havia artefato Windows publicado com essa correção; por isso foi necessário compilar a `main`.

> O PR #7292 relata `VK_ERROR_INVALID_EXTERNAL_HANDLE` em uma RX 5700 XT. Nesta máquina, com a RX 9060 XT, o mesmo problema apareceu como `VK_ERROR_OUT_OF_DEVICE_MEMORY` (`-2`). A causa é a mesma: alocação única maior que o limite do driver.

---

5. O que foi feito, passo a passo

5.1 Backups e estado inicial

Antes de alterar o runtime, os logs antigos foram preservados:

- `work\run.v013.out.log`
- `work\run.v013.err.log` (contém o crash original)
- `work\run.agconly.out.log`
- `work\run.agconly.err.log` (contém o erro 127 da tentativa com apenas o driver AGC)

Backups das libs:

- `work\libs.v0.1.3.full.bak` — **backup limpo da v0.1.3**, recomendado para rollback.
- `work\libs.v0.1.3.bak` — backup parcial feito depois da primeira troca do AGC; **não usar sozinho para rollback**.
- `work\libs\libSceAgcDriver.prx.v0.1.3.bak` — driver AGC antigo, 15.906.843 bytes.

> Na primeira tentativa, o backup completo foi feito em duas etapas. Para o futuro, faça o backup **antes** de qualquer cópia. O backup limpo final foi criado copiando o parcial e restaurando o driver antigo.

5.2 Ferramentas

O CMake e o Python já existiam. Foram instalados/extraídos:

- **Ninja 1.13.2** via winget, escopo de usuário: `powershell winget install --id Ninja-build.Ninja -e --scope user --accept-package-agreements --accept-source-agreements --silent --disable-interactivity`
- **Git portátil** (o winget do Git exigia elevação e falhou):
- Download: `PortableGit-2.55.0.5-64-bit.7z.exe`
- Extração sem admin: `powershell Start-Process -FilePath 'C:\Users\italo\AppData\Local\Temp\opencode\PortableGit-2.55.0.5-64-bit.7z.exe' ` -ArgumentList '-y','-oC:\tools\PortableGit' -Wait -NoNewWindow`
- **WinLibs GCC 15.2.0** (`x86_64-ucrt-posix-seh`, release `15.2.0posix-14.0.0-ucrt-r7`):
- URL usada: `https://github.com/brechtsanders/winlibs_mingw/releases/download/15.2.0posix-14.0.0-ucrt-r7/winlibs-x86_64-posix-seh-gcc-15.2.0-mingw-w64ucrt-14.0.0-r7.zip`
- Extraído em `C:\tools\winlibs` (contém `mingw64\bin`).

PATH usado em todas as etapas de build/teste:

```
$env:PATH =
"C:\tools\winlibs\mingw64\bin;C:\tools\PortableGit\cmd;C:\tools\PortableGit\usr\bin;C:\Users\italo\AppData\Local\Microsoft\WinGet\Packages\Ninja-build.Ninja_Microsoft.Winget.Source_8wekyb3d8bbwe;$env:PATH"
```

O Git for Windows completo é necessário porque o build do libusb usa Bash e `cygpath`. Um MinGit sem Bash não é suficiente.

5.3 Clonar a `main` com submódulos

```
git -c core.longpaths=true clone --depth 1 --recurse-submodules --shallow-submodules --jobs 4 `
  https://github.com/boykopovar/AnyPS5.git D:\AnyPS5-main
```

Commit usado e verificado:

```
a0fb7c26a7969d0ae2b18aee923e43a30d133898
2026-10-24 00:20:00 +0200
Merge pull request #8968 from Chuk49/fix/agc-capture-skip-float-probe
```

Verificação de que a correção está presente:

```
Select-String -LiteralPath 'D:\AnyPS5-main\core\libs\prx\libSceAgcDriver\Graphics\src\GuestBufferMemory.cpp'
`
  -Pattern 'importMaxBytes|importPieces|SetHostImportMaxBytes'
```

Resultado esperado: linhas com `importMaxBytes`, `importPieces` e `SetHostImportMaxBytes` (por volta das linhas 1529, 1643 e 3511).

> O clone foi shallow para economizar tempo/espaço. A `main` é móvel; **sempre registre o commit** usado. Para reprodução estrita, faça checkout desse commit ou um clone completo.

5.4 Configurar o CMake

O download do FFmpeg durante o configure falha com o CMake do WinLibs se o certificado CA não for informado. Foi usado o CA bundle do PortableGit:

```
$env:SSL_CERT_FILE = 'C:\tools\PortableGit\mingw64\etc\ssl\certs\ca-bundle.crt'
$env:CURL_CA_BUNDLE = $env:SSL_CERT_FILE
```

Comando de configure usado:

```
cmake -S D:\AnyPS5-main -B D:\AnyPS5-main\build -G Ninja `
  -DCMAKE_BUILD_TYPE=Release `
  -DCMAKE_C_COMPILER='C:/tools/winlibs/mingw64/bin/gcc.exe' `
  -DCMAKE_CXX_COMPILER='C:/tools/winlibs/mingw64/bin/g++.exe' `
  -DCMAKE_MAKE_PROGRAM='C:/Users/italo/AppData/Local/Microsoft/WinGet/Packages/Ninja-
build.Ninja_Microsoft.Winget.Source_8wekyb3d8bbwe/ninja.exe' `
  -DBUILD_TESTING=OFF `
  -DPython3_EXECUTABLE='C:/Users/italo/AppData/Local/Programs/Python/Python312/python.exe'
```

Resultado: configure concluído em 107,1 s, geração em 4,7 s.

5.5 Compilar as libs

```
cmake --build D:\AnyPS5-main\build --target libs --parallel 8
```

- Saída: `D:\AnyPS5-main\build\core\libs\libs`
- 2506 etapas; término com `exit 0`; ≈16 min.
- Com `BUILD_TESTING=OFF`, o alvo `libs` é `EXCLUDE_FROM_ALL`; **é obrigatório pedir `--target libs` explicitamente**. Um build padrão não produz os `.prx`.

Copiar as DLLs de runtime do toolchain para junto das libs:

```
$toolchainBin = 'C:\tools\winlibs\mingw64\bin'
$dst = 'D:\AnyPS5-main\build\core\libs\libs'
foreach ($n in 'libgcc_s_seh-1.dll','libstdc++-6.dll','libwinpthread-1.dll') {
  Copy-Item -LiteralPath (Join-Path $toolchainBin $n) -Destination $dst -Force
}
```

5.6 Deploy no workspace do jogo

Importante: a primeira tentativa copiou apenas `libSceAgcDriver.prx`. O jogo falhou ao carregar `libSceVideoOut.prx`:

```
Failed to load module: D:\AnyPS5-AstroBot\work\libs\libSceVideoOut.prx
GetLastError: 127 - The specified procedure could not be found.
```

Isso acontece porque o driver AGC novo importa símbolos que as libs antigas não exportam (ou o conjunto não é compatível). A solução foi copiar **todas as libs** do mesmo build:

```
$work = 'D:\AnyPS5-AstroBot\work'
$src  = 'D:\AnyPS5-main\build\core\libs\libs'

# Fazer o backup ANTES de copiar (no futuro)
# Copy-Item -LiteralPath "$work\libs" -Destination "$work\libs.v0.1.3.full.bak" -Recurse

Copy-Item -Path "$src\*" -Destination "$work\libs" -Recurse -Force
```

O que **não** foi alterado nesta sessão:

- `work\app.exe`
- `work\app0`
- configuração do jogo
- conversão prévia de `libSceNpCppWebApi.prx` para `app0\sce_module\libSceNpCppWebApi.prx.guest.prx` (já existia antes)

5.7 Teste

Backup dos logs atuais, encerramento de instâncias antigas e execução com timeout:

```
$work = 'D:\AnyPS5-AstroBot\work'
Get-Process app -ErrorAction SilentlyContinue | Stop-Process -Force

$env:ANYPS5_GPU = 'Radeon'
$env:ANYPS5_LANGUAGE = '17'
$env:APS5_TRACE_DIR = Join-Path $work 'traces'

$p = Start-Process -FilePath (Join-Path $work 'app.exe') -WorkingDirectory $work `
  -RedirectStandardOutput (Join-Path $work 'run.out.log') `
  -RedirectStandardError  (Join-Path $work 'run.err.log') -PassThru

if (-not $p.WaitForExit(180000)) {
  'Ainda rodando após 180 s; encerrando o teste'
  Stop-Process -Id $p.Id -Force
}
```

Verificação de erros fatais:

```
Select-String -Path "$work\run.out.log","$work\run.err.log" `
  -Pattern 'what\(\):|VK_ERROR|OUT_OF_DEVICE_MEMORY|device lost|Failed to load|abort|terminate
called|0xc0000409' `
  -CaseSensitive:$false
```

Resultado: nenhuma ocorrência. Linhas relevantes do sucesso:

```
LevelDocument Loaded: title_controller_ship [title]
GAME: LoadLevelResources: title_controller_ship : 5.320 sec.
[mouse] video driver windows: guest 3840x2160 presented at 2201x1238, delta scale 1.7447 x 1.7447
```

O processo ainda estava vivo após 180 s e foi encerrado manualmente. Portanto, o teste comprova que o crash foi eliminado e que o jogo chegou à tela/título, mas **não** é uma validação completa de gameplay.

---

6. Resultado e evidências

Item	v0.1.3	`main` `a0fb7c2`
`libSceAgcDriver.prx`	15.906.843 bytes	17.104.761 bytes
SHA-256	`0B8EFCAC34728314C34FAB8CC7D086C2E642B058ED3DDB4C5918309F6E9EA304`	`133AD9421B5DED833066E18DEC86B4F2D3DABFD36DD43C1F5EDB22D6F1EB2170`
Heap mirror de 3,15 GiB	aborta com `VK_ERROR_OUT_OF_DEVICE_MEMORY`	não aborta
Carregamento do título	não chegava	`title_controller_ship` carregado
Apresentação	não chegava	2201×1238
Erros fatais em 180 s	sim	nenhum

Linhas de orçamento de import ainda aparecem no log, mas não são fatais. Exemplo:

```
[gpu] host import budget: 3545 MiB live of 4084 MiB (the default; host heap 1 reports a budget of 7505 MiB and a usage of 2893 MiB); 16 imports (31744 MiB) refused so far, 0 of them remembered, last 0x500000000+0x7c000000 by build
```

Isso é o comportamento normal de fallback: alguns imports são recusados por orçamento e o driver usa cópias/outros caminhos. O que não pode aparecer é `VK_ERROR_OUT_OF_DEVICE_MEMORY` seguido de `what()`/abort.

---

7. Artefatos e caminhos

Runtime do jogo

- Executável: `D:\AnyPS5-AstroBot\work\app.exe`
- Libs novas: `D:\AnyPS5-AstroBot\work\libs`
- Backup limpo v0.1.3: `D:\AnyPS5-AstroBot\work\libs.v0.1.3.full.bak`
- Backup parcial (não usar sozinho): `D:\AnyPS5-AstroBot\work\libs.v0.1.3.bak`
- Driver antigo: `D:\AnyPS5-AstroBot\work\libs\libSceAgcDriver.prx.v0.1.3.bak`
- Logs do teste bem-sucedido: `D:\AnyPS5-AstroBot\work\run.out.log`, `run.err.log`
- Log do crash antigo: `D:\AnyPS5-AstroBot\work\run.v013.err.log`

Build

- Fonte: `D:\AnyPS5-main`
- Build: `D:\AnyPS5-main\build`
- Libs compiladas: `D:\AnyPS5-main\build\core\libs\libs`
- Log de configure: `C:\Users\italo\AppData\Local\Temp\opencode\configure-main.log`
- Log de build: `C:\Users\italo\AppData\Local\Temp\opencode\build-libs.out.log`
- Log do build final: `C:\Users\italo\AppData\Local\Temp\opencode\build-libs-final.log`

Ferramentas

- GCC: `C:\tools\winlibs\mingw64\bin`
- Git portátil: `C:\tools\PortableGit`
- Ninja: `C:\Users\italo\AppData\Local\Microsoft\WinGet\Packages\Ninja-build.Ninja_Microsoft.Winget.Source_8wekyb3d8bbwe\ninja.exe`
- CMake do WinLibs: `C:\tools\winlibs\mingw64\bin\cmake.exe` (4.3.1)
- CMake do sistema: `C:\Program Files\CMake\bin\cmake.exe` (4.4.4)

---

8. Checklist para próximos portes

A. Diagnosticar

1. Rodar o jogo com `stdout` / `stderr` redirecionados para arquivos.
2. Procurar por `what()`, `VK_ERROR`, `OUT_OF_DEVICE_MEMORY`, `device lost`, `Failed to load`, `0xc0000409`.
3. Anotar o tamanho da alocação que falhou e o `maxMemoryAllocationSize` do driver.
4. Comparar a versão do emulador com a `main`; a correção pode já existir.
5. Procurar PRs/commits por arquivo e por mensagem:
   - `https://api.github.com/repos/boykopovar/AnyPS5/commits?path=core/libs/prx/libSceAgcDriver/Graphics/src/GuestBufferMemory.cpp&per_page=15`
   - Buscar por “largest allocation”, “heap mirror”, “import pieces”.
6. Confirmar no fonte se a correção está presente (`importMaxBytes`, `importPieces`, `SetHostImportMaxBytes`).

B. Compilar

7. Instalar Git com Bash, Ninja, GCC MinGW-w64 com POSIX threads e CMake ≥3.22.
8. Clonar a `main` com submódulos; registrar o commit.
9. Configurar com `-DBUILD_TESTING=OFF`, `-DPython3_EXECUTABLE=...` e `SSL_CERT_FILE` para o FFmpeg.
10. Compilar com `cmake --build build --target libs --parallel <n>`.
11. Copiar `libgcc_s_seh-1.dll`, `libstdc++-6.dll` e `libwinpthread-1.dll` para junto das libs.

C. Implantar

12. Fazer backup **da pasta `libs` inteira** antes de mexer.
13. Copiar **todas** as libs do mesmo build, não só o driver alterado.
14. Manter `app.exe` e `app0` intactos, a menos que o relinker/tradutor também tenha mudado.
15. Se aparecer `GetLastError 127`, há mistura de libs ou import não resolvido: alinhar o conjunto completo e, se necessário, re-relinkar o `app.exe` com as mesmas libs.

D. Testar

16. Rodar com `ANYPS5_GPU=Radeon` e `ANYPS5_LANGUAGE=17` (ajustar ao jogo).
17. Esperar tempo suficiente para passar do ponto do crash; usar timeout e encerrar manualmente se necessário.
18. Verificar ausência de erros fatais e presença de evidências de renderização (`LevelDocument Loaded`, `presented at`, texturas).
19. Para validação real, interagir com o jogo e rodar por mais tempo; 180 s na tela de título não é gameplay completo.

E. Rollback

20. Encerrar o jogo.
21. Substituir `libs` pelo backup limpo.
22. Conferir tamanho/hash do driver antigo.
23. Rodar novamente para confirmar que o runtime voltou ao estado anterior.

---

9. Armadilhas e lições aprendidas

- **Nunca misture libs de builds diferentes.** O erro 127 em `libSceVideoOut.prx` foi causado exatamente por isso.
- **O driver AGC sozinho não basta.** O conjunto de PRXs precisa ser coerente.
- **A release v0.1.3 é antiga.** A correção estava só na `main`; não havia artefato Windows publicado com ela.
- **Limite por alocação ≠ VRAM livre.** Havia VRAM de sobra; o driver recusava uma alocação individual >2 GiB.
- **Flags de heap/import não corrigem alocação única >2 GiB.** É necessário dividir a faixa em pedaços.
- **Git via winget pode exigir admin.** PortableGit resolveu sem elevação.
- **FFmpeg + CMake do WinLibs pode falhar com erro 60 de certificado.** Definir `SSL_CERT_FILE` e `CURL_CA_BUNDLE`.
- **`BUILD_TESTING=OFF` não compila `libs` por padrão.** Pedir `--target libs`.
- **Clone shallow é prático, mas a `main` muda.** Registre o commit e os hashes dos artefatos.
- **Teste com timeout não é saída limpa.** "Ainda rodando após 180 s" significa que não crashou; não significa que o jogo foi validado até o fim.
- **O `AdapterRAM` do WMI pode mentir.** Use os heaps do Vulkan/log para dimensionar memória.
- **Recusas de host import não são necessariamente fatais.** Verifique o tipo de erro e se o jogo continua.

---

10. Rollback

Para voltar ao estado v0.1.3:

```
$work = 'D:\AnyPS5-AstroBot\work'
Get-Process app -ErrorAction SilentlyContinue | Stop-Process -Force

# Preserve o estado atual, se quiser
if (Test-Path "$work\libs") {
  Rename-Item -LiteralPath "$work\libs" -NewName 'libs.main-20261023'
}

Copy-Item -LiteralPath "$work\libs.v0.1.3.full.bak" -Destination "$work\libs" -Recurse
```

Conferir:

```
Get-Item "$work\libs\libSceAgcDriver.prx" |
  Select-Object Length, LastWriteTime
Get-FileHash "$work\libs\libSceAgcDriver.prx" -Algorithm SHA256
# Esperado: 15.906.843 bytes
# SHA-256: 0B8EFCAC34728314C34FAB8CC7D086C2E642B058ED3DDB4C5918309F6E9EA304
```

Não use `libs.v0.1.3.bak` sozinho para rollback: ele foi criado depois da primeira troca do AGC e contém o driver novo. Use `libs.v0.1.3.full.bak`.

---

11. Referências

- AnyPS5: https://github.com/boykopovar/AnyPS5
- PR #7292: https://github.com/boykopovar/AnyPS5/pull/7292
- PR #7647: https://github.com/boykopovar/AnyPS5/pull/7647
- Commit da `main` usada: `a0fb7c26a7969d0ae2b18aee923e43a30d133898`
- Merge commit do PR #7292: `b061d2a621cf1a573532341197f4e3f04e05a085`
- Merge commit do PR #7647: `2f83cc782f5b71976f7a91fb58cb801177f2abd2`
- Documentação de build: `D:\AnyPS5-main\docs\dev\BUILD.md`
- Documentação de variáveis: `D:\AnyPS5-main\docs\dev\ENVIRONMENT.md`
- Log do crash original: `D:\AnyPS5-AstroBot\work\run.v013.err.log` (linha 313)
- Log da tentativa com apenas AGC: `D:\AnyPS5-AstroBot\work\run.agconly.err.log`
- Log do teste bem-sucedido: `D:\AnyPS5-AstroBot\work\run.out.log` e `run.err.log`

---

12. Limitações deste teste

- O teste de 180 s chegou ao título (`title_controller_ship`) e apresentou frames, mas **não validou gameplay completo**, estabilidade de longa duração, áudio, controles ou desempenho.
- O build foi feito com `BUILD_TESTING=OFF`; a suíte de testes do AnyPS5 não foi executada.
- O `app.exe` e o `app0` não foram reconstruídos; o jogo continua usando o relinker/conversão da v0.1.3.
- A `main` contém outras mudanças além dos PRs #7292/#7647; o conjunto completo de libs inclui essas mudanças. Se aparecer regressão, será necessário bissectar ou re-relinkar com um conjunto mais controlado.
- O procedimento foi validado para **ASTRO BOT** `PPSA21567` nesta máquina. Outros jogos podem exigir PRs adicionais ou ajustes de orçamento de memória.

Apêndice A — Sequência completa resumida

```
# 1) Ferramentas (uma vez)
# Ninja via winget; Git Portable; WinLibs GCC 15.2.0; CMake/Python já instalados.

$env:PATH =
"C:\tools\winlibs\mingw64\bin;C:\tools\PortableGit\cmd;C:\tools\PortableGit\usr\bin;C:\Users\italo\AppData\Lo
cal\Microsoft\WinGet\Packages\Ninja-build.Ninja_Microsoft.Winget.Source_8wekyb3d8bbwe;$env:PATH"
$env:SSL_CERT_FILE = 'C:\tools\PortableGit\mingw64\etc\ssl\certs\ca-bundle.crt'
$env:CURL_CA_BUNDLE = $env:SSL_CERT_FILE

# 2) Fonte
git -c core.longpaths=true clone --depth 1 --recurse-submodules --shallow-submodules --jobs 4 `
  https://github.com/boykopovar/AnyPS5.git D:\AnyPS5-main

# 3) Configure
cmake -S D:\AnyPS5-main -B D:\AnyPS5-main\build -G Ninja `
  -DCMAKE_BUILD_TYPE=Release `
  -DCMAKE_C_COMPILER='C:/tools/winlibs/mingw64/bin/gcc.exe' `
  -DCMAKE_CXX_COMPILER='C:/tools/winlibs/mingw64/bin/g++.exe' `
  -DCMAKE_MAKE_PROGRAM='C:/Users/italo/AppData/Local/Microsoft/WinGet/Packages/Ninja-
build.Ninja_Microsoft.Winget.Source_8wekyb3d8bbwe/ninja.exe' `
  -DBUILD_TESTING=OFF `
  -DPython3_EXECUTABLE='C:/Users/italo/AppData/Local/Programs/Python/Python312/python.exe'

# 4) Build
cmake --build D:\AnyPS5-main\build --target libs --parallel 8

# 5) DLLs de runtime
foreach ($n in 'libgcc_s_seh-1.dll','libstdc++-6.dll','libwinpthread-1.dll') {
  Copy-Item -LiteralPath "C:\tools\winlibs\mingw64\bin\$n" `
    -Destination 'D:\AnyPS5-main\build\core\libs\libs' -Force
}

# 6) Backup e deploy
$work = 'D:\AnyPS5-AstroBot\work'
Copy-Item -LiteralPath "$work\libs" -Destination "$work\libs.v0.1.3.full.bak" -Recurse
Copy-Item -Path 'D:\AnyPS5-main\build\core\libs\libs\*' -Destination "$work\libs" -Recurse -Force

# 7) Teste
Get-Process app -ErrorAction SilentlyContinue | Stop-Process -Force
$env:ANYPS5_GPU = 'Radeon'
$env:ANYPS5_LANGUAGE = '17'
$env:APS5_TRACE_DIR = "$work\traces"
$p = Start-Process -FilePath "$work\app.exe" -WorkingDirectory $work `
  -RedirectStandardOutput "$work\run.out.log" `
  -RedirectStandardError  "$work\run.err.log" -PassThru
if (-not $p.WaitForExit(180000)) { Stop-Process -Id $p.Id -Force }
```

```
# 8) Verificação
Select-String -Path "$work\run.out.log","$work\run.err.log" `
  -Pattern 'what\(\):|VK_ERROR|OUT_OF_DEVICE_MEMORY|device lost|Failed to load|0xc0000409' `
  -CaseSensitive:$false
```